StoreSkuTracker Developer Handoff

Prepared on 2026-06-16

Project Overview

StoreSkuTracker is a small Flask + SQLite web app used to track store-level SKU shelf counts, expiring items, and current stock health.

The app has two main pages: /dashboard for live stock updates and /manage for managing stores, SKUs, and store-to-SKU assignments.

The current production deployment is on PythonAnywhere Free under the account/site: https://trujoescoffee.pythonanywhere.com

Current Tech Stack

- Backend: Flask in app.py
- Database: SQLite at instance/stock_tracker.sqlite3
- Frontend: Jinja templates in templates/, JavaScript in static/app.js, CSS in static/styles.css
- Deployment target: PythonAnywhere Free plan
- PythonAnywhere project path: /home/trujoescoffee/StoreSkuTracker
- PythonAnywhere virtualenv: /home/trujoescoffee/.virtualenvs/storesku

Main Features Implemented

- Dashboard shows current stock status for each active store/SKU assignment.
- Employees can update shelf count and expiring count inline from the dashboard.
- Manage page can add/delete stores and SKU names.
- Manage page can assign SKUs to stores.
- Each store/SKU assignment has a recommended shelf count.
- Recommended shelf count can be edited directly from the assignment list.
- Dashboard includes a manager-friendly note explaining all status and color conditions.
- Dashboard and recent visits are mobile-friendly: on phones, tables become stacked cards.
- Historical visit records are preserved even when stores/SKUs are removed from the active catalog.

Status and Color Rules

- Healthy: current shelf count is above 50% of the recommended count and no items are expiring.
- Unhealthy: current shelf count is 20% to 50% of the recommended count and no items are expiring.
- Critical: at least 1 item is expiring, shelf count is below 20%, no count exists, or last visit is older than 72 hours.
- Rows with visits older than 72 hours are tinted red and marked Critical.

Database Tables

- stores: active store list.
- skus: active SKU list.
- store_skus: active store-to-SKU mappings, including recommended_count.
- stock_visits: historical visit/update records. This table stores store and SKU names as text so history remains readable after catalog changes.

Important Behavior Notes

- The app no longer repopulates active stores/SKUs from stock_visits on startup. This was fixed because deleted stores were reappearing after reload.
- Existing store/SKU mappings received a default recommended_count of 10 when the migration was added.
- Adding the same store/SKU mapping again from /manage updates its recommended count instead of creating a duplicate.
- Inline dashboard saves create a new stock_visits row. The dashboard then calculates current status from the latest visit per store/SKU pair.

Local Development Commands

- Create virtualenv: python3 -m venv .venv
- Activate virtualenv: source .venv/bin/activate
- Install dependencies: pip install -r requirements.txt
- Run locally: python app.py
- Local URL: http://127.0.0.1:8001
- Syntax check used during development: .venv/bin/python -m py_compile app.py

PythonAnywhere Deployment Steps Completed

- Code was uploaded to PythonAnywhere via ZIP.
- ZIP was extracted into /home/trujoescoffee/StoreSkuTracker.
- Virtualenv storesku was created.
- Dependencies were installed from requirements.txt.
- A manual Flask web app was configured from the PythonAnywhere Web tab.
- WSGI path was configured to import app from /home/trujoescoffee/StoreSkuTracker.
- Static files mapping was configured: URL /static/ -> /home/trujoescoffee/StoreSkuTracker/static.
- The app was reloaded and confirmed running at https://trujoescoffee.pythonanywhere.com.

PythonAnywhere WSGI Configuration

```
import sys

path = '/home/trujoescoffee/StoreSkuTracker'
if path not in sys.path:
    sys.path.insert(0, path)

from app import app as application
```

Current Production Paths

- Application code: /home/trujoescoffee/StoreSkuTracker
- SQLite database: /home/trujoescoffee/StoreSkuTracker/instance/stock_tracker.sqlite3
- Static files: /home/trujoescoffee/StoreSkuTracker/static
- Templates: /home/trujoescoffee/StoreSkuTracker/templates
- Virtualenv: /home/trujoescoffee/.virtualenvs/storesku

Backup Guidance

Because the app uses SQLite on the PythonAnywhere Free plan, the database file should be backed up periodically. A simple manual backup command is:

```
cp ~/StoreSkuTracker/instance/stock_tracker.sqlite3 ~/stock_tracker_backup.sqlite3
```

Recommended Next Improvements

- Add login/password protection before wider employee rollout. Currently anyone with the URL can edit data.
- Move SECRET_KEY to an environment variable instead of hardcoding it.
- Add user roles later if managers and employees should have different permissions.
- For more users or heavier simultaneous updates, consider moving from SQLite to MySQL/Postgres.
- Add automated database backups if this becomes business-critical.
- Clean up committed local-only files such as .venv, __pycache__, and generated database copies before maintaining the GitHub repo.

Files Future Developer Should Review First

- app.py: Flask routes, SQLite schema, migrations, and status rules.
- templates/index.html: dashboard UI and manager status note.
- templates/manage.html: store/SKU management and recommended count editing.
- static/app.js: inline dashboard AJAX save and client-side filtering.
- static/styles.css: desktop, mobile, status, and card/table styling.
- README.md: run instructions and project summary.